

MUSIK 2.0: MIDI-Utilized System Interface for Keyboards

Hannah Bialik, *ECE/CS*, Adin Dantowitz, *MAE*, Jeremy Hui, *ECE/CS*, Daniel Lee, *MAE*,
Christopher Santos, *MAE*, Noa Strupinsky, *ECE/CS*, and Hans Wassermann, *ECE*

Abstract—The goal of MUSIK is to simplify the piano playing experience by eliminating the need to read complex sheet music, making the piano more accessible and easier to play. It enriches the piano-playing experience by integrating visual cues and timing feedback that augment the conventional auditory experience. The enhancement is achieved through an augmented reality display of lanes where piano keys move down the lanes onto the keyboard. Utilizing a MIDI (Musical Instrument Digital Interface) file, our display guides the user by showing which notes to play and the precise timing for each. The experience is enabled through an ergonomically designed headset modeled after the Google Cardboard. As such, anyone with a piano can get started in learning to play, bringing greater accessibility to music education.

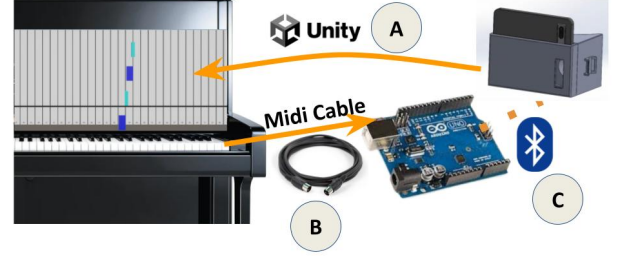


Fig. 1. Physical and Digital Element Connections

CONTENTS

I	Introduction	1
I-A	Digital Element	1
I-B	Physical Prototype Element	1
I-C	Social Relevance and Application Element	2
II	Equipment and Methodology	2
II-A	Headset Design	2
II-A1	Design Overview	2
II-A2	Key Considerations	2
II-A3	Manufacturing and Assembly	3
II-B	Microcontroller Communication Protocol	3
II-C	Game Display	3
II-C1	Virtual screen	3
II-C2	Augmented reality	4
II-D	Accuracy Computation	4
II-D1	MIDI2Tiles Procedure	5
II-D2	Input Queue	5
III	Results and Discussion	6
III-A	Future Work and Improvements	6
IV	Conclusions	7
	References	7

I. INTRODUCTION

OFTENTIMES when learning to play the piano, the disconnect between reading sheet music notation and the music that is created can create a large obstacle in the piano playing experience. A popular solution to this problem is to learn from a music teacher, but they can often be hard to find or expensive to learn from, motivating the need for at

home learning systems. An existing design that inspired this project was MUSIK 1.0, where the implementation consisted of a projector display that would realign according to the keys. We expand upon this previous design through creating an AR system, in order to reduce the overhead towards learning piano through having a self-contained product with non-intrusive elements. With MUSIK 2.0, learning to play the piano becomes much easier and accessible, allowing anyone to have a more fun experience while learning

A. Digital Element

The core implementation of MUSIK 2.0 is programming intensive and aims to fulfill our objectives. The project's code base is divided into two main platforms: high-level visualizer scripting in Unity and low-level microcontroller accuracy processing in the Arduino UNO R3. The Arduino is going to compare the user generated MIDI file from their piano playing and the sample MIDI file and generate an accuracy score. The Unity scripts will run on the user's phone from a downloaded APK file that has the visualizer and will connect to an additional bluetooth module on the Arduino. The purpose of the bluetooth connection is to start a global timer when the user starts the game and to send the user their accuracy score once they've completed playing the song (from the Arduino to Unity).

B. Physical Prototype Element

A basic overview of our physical prototype is shown in Fig. 1. Our prototype consists of two main physical components: the phone/headset and the Arduino. We link these elements together by connecting a MIDI cable from the keyboard to the Arduino, and then connecting the Arduino to the

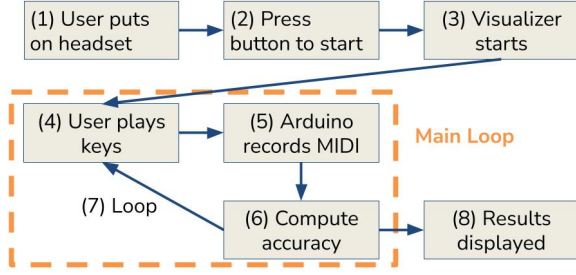


Fig. 2. Flow Chart of Procedure

Unity APK on the phone with bluetooth. With our limited physical components, our originally cumbersome design with a projector, computer, and LED strip has been greatly reduced to be as non-intrusive to the user as possible.

C. Social Relevance and Application Element

Our piano teaching techniques in MUSIK 2.0 help engage children and adults alike who are new to playing piano. Introductory piano learners no longer have to first understand musical notation and instead can learn how to play their favorite songs visually. MUSIK 2.0 also makes music education more accessible to those who do not have the opportunity to learn from a piano teacher. This also encourages self-learning and home education for those who want to learn to play the piano in their own free time.

II. EQUIPMENT AND METHODOLOGY

To introduce the design of the system and methodology used when developing the entire prototype, we first present the flowchart shown in Fig. 2, which follows the procedure in which a user will employ when using the system. We will now introduce the components of the prototype that follow this flowchart: the headset the user puts on, communication with the visualizer to start and sending timing messages, the visualizer itself that displays the MIDI information in an AR display to the user, and the main program that drives the computation.

A. Headset Design

The headset aims to combine the accessibility and low cost of the Google Cardboard [3] with the immersive experience found in virtual reality headsets. To accomplish this and deliver a reliable and easily manufactured prototype, we developed the headset to address three core areas: design of the headset through computer-aided design (CAD), evaluation of factors such as dimensioning and part selection, and a manufacturing process centered around 3D printing.

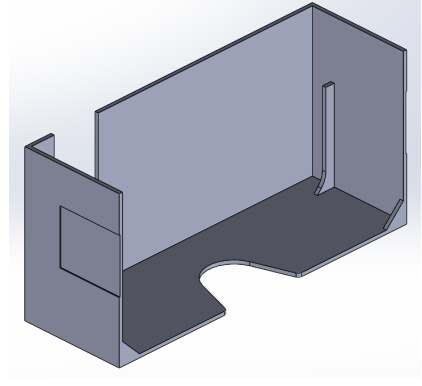


Fig. 3. Headset base

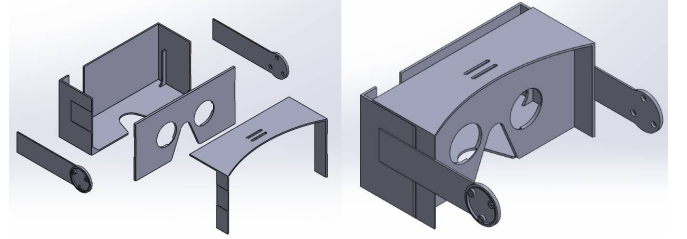


Fig. 4. Assembled headset. Left: Exploded view. Right: Collapsed view.

1) *Design Overview:* The headset was modeled in SolidWorks and split into four unique components that were dimensioned with digital calipers and designed with 3D printing in mind.

- 1) The base (Fig. 3) that securely houses the smartphone and has an opening for its camera.
- 2) The facial interface that has two circular openings for the lenses with a diameter of 34.1 mm separated by a pupillary distance of 64 mm.
- 3) The top cover to prevent external light from interfering with the smartphone display.
- 4) The pair of strap mounts that were designed to match the dimensions of the head strap's circular connection mechanism.

An assembled view of the headset, seen in Fig. 4, illustrates how these parts fit together.

2) *Key Considerations:* With the goals for the headset in mind, we considered several important factors involved in its development. Component-wise, a lot of the geometry was inspired by the Google Cardboard [3], as its simplistic yet functional design would allow for both reliability and streamlined manufacturing. The headset parts also were designed to have thicknesses ranging from 2-3 mm, further reducing print-time and filament usage. Furthermore, contact surfaces between components were added to ensure adequate adhesion during assembly. For filament, we selected polylactic acid (PLA) for its low cost and effective adhesion with super glue.

In addition to the body of the headset that we designed, we also selected and incorporated other parts that would enhance the headset's functionality. We used biconvex lenses for the facial interface to help emulate stereoscopic vision.

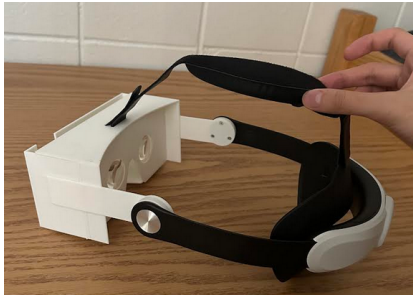


Fig. 5. Completed prototype

To incorporate these, the distance between the facial interface and the phone screen was dimensioned to be around 45 mm to match the lenses' focal distance [8]. We also chose an existing head strap—similar to the ones seen on virtual reality headsets—so the device would remain secure when the user moves their head.

3) *Manufacturing and Assembly*: Before manufacturing the headset, multiple test prints were required to get an idea of the required dimensions and tolerances required for the final prototype, especially between the strap mounts and the head strap. Once our test print was functional, its features were translated onto its corresponding CAD model to be incorporated in our final design.

After finalizing our design, all components were printed, assembled using super glue, and further strengthened by smearing an epoxy bonding agent over edges and grooves. Finally, the lenses were snap-fitted into the facial interface, and the head strap was screwed onto the headset. The fully assembled prototype is illustrated in Fig. 5.

B. Microcontroller Communication Protocol

In order to facilitate communication and synchronization between the Unity game display and the Python program that facilitates the MIDI keyboard input, Bluetooth technology using the HC-05 device was chosen to send and receive messages between both parties. In order to set up the communication, the following line was placed at the start of the Arduino script,

```
SoftwareSerial BT(10, 11);
```

where 10 represents the receiving pin and 11 represents the transmitting pin. Then throughout the program, the following will execute to act as a proxy between the Unity program and the Python program, where if

```
Serial.available() > 0
```

(meaning data from Python is available to be sent), Arduino will call

```
String data = Serial.readStringUntil('\n');
BT.println(data);
```

to extract the next line, and to send the data.

Similarly, if

```
BT.available() > 0
```

(meaning data from Unity is available to be received), Arduino will call

```
String data = BT.readStringUntil('\n');
Serial.println(data);
```

to extract the next line, and to receive the data.

For the Python program, in order to transmit messages, the following line was used to setup the Arduino,

```
arduino = serial.Serial(arduino_port,
baud_rate, timeout=1)
```

where `arduino_port` is set to 'COM_4' and the `baud_rate` is set to 115200. Then whenever a message needs to be sent, the `arduino.write()` function is used, making sure to first encode the message in UTF-8 format to match the encoding of Arduino, as well as appending a newline character at the end to delimit messages.

With this Arduino program running in the background, we can then have Unity send a start message to the Arduino to signal to start the game. Then while the program is executing, the timing messages can be sent to the system, as well as the final score to be displayed once the game over message is received.

C. Game Display

The goals of the display are to clearly communicate which keys the user should press at any given moment and to provide real-time feedback on their performance. Developing the Augmented Reality (AR) display involved two main tasks. The first was creating the game screen itself, including a visual representation of the MIDI song file and user interface elements such as live feedback messages and a "game over" display. The second task was integrating the virtual screen into the physical world, which required ensuring Android compatibility, creating a stereoscopic view to be integrated with the headset, and aligning the virtual display with the user's physical keyboard.

1) *Virtual screen*: The virtual screen was created using Unity. The key components of this display are its visual representation of the song, and the live feedback that it provides.

a) *MIDI visualizer*: Songs are input into our program in MIDI format, which serves as a digital equivalent to sheet music. The MIDI protocol conveys information as state changes within the instrument. It assigns numerical values to musical notes, enabling identification of the corresponding note for each event. For instance, engaging and disengaging middle C on a piano triggers two distinct messages for the press and release, assigned the note number 60.

The DryWetMidi [2] Unity asset is used to parse the MIDI file, extracting the name, start time, and end time of each

note, and storing this data in an array. It also creates a TempoMap object that stores the Pulses Per Quarter Note (PPQN) and a list of tempo changes throughout the song. In the visualizer, each column corresponds to a specific piano key. These columns are independent Unity objects, each of which contains a script that filters the array, selecting only the notes matching its assigned piano key. As a result, each column maintains a list of start and end times for its relevant notes. However, these start and end times are provided as pulses — an abstract unit of time — so they must be converted to seconds in order to be useful. The following equation is used to convert the time from ticks to seconds:

$$seconds = \frac{Ticks * Tempo}{PPQN * 1,000,000}$$

where Tempo is the tempo in microseconds per quarter note and Ticks is the MIDI time value for the note. The TempoMap object is used to retrieve the PPQN and tempo at each tick during the conversion. When the user presses the play button, the start time of the song is stored. Using the start time as a relative zero, the columns dynamically spawn and remove notes at the appropriate times during the game. An example of this is shown in Fig. 6.

As previously described, each piano key is associated with an independent Unity object. These objects were manually aligned with a background image consisting of 52 labeled columns, where each column corresponds to a white key. Objects representing white keys were positioned within the appropriate columns, while objects representing black keys were placed along the lines separating the columns. As seen in Fig. 6, we decided to use one color for regular notes and a different color for sharp notes, to help the user better differentiate between these types of notes.

b) User interface: As discussed earlier, in order to provide feedback on the user's playing, as calculated on the Arduino, we decided to use a Bluetooth module to communicate between the Arduino and the Unity app. This feedback was displayed in the Unity game screen using TextMeshPro objects, as shown in Fig. 6. When the program starts, a welcome message appears at the center of the screen, while notifications about the Bluetooth connection status are shown in the corner. Once the program receives a "start" message via Bluetooth, the MIDI visualizer begins, and feedback messages from the Arduino are displayed in the corner throughout gameplay. When the game concludes, the screen presents the player's final score, as shown in Fig. 7.

2) Augmented reality: A variety of assets were utilized to develop an app that integrates augmented reality. We used Unity with Vuforia and Google Cardboard packages.

a) AR/VR: To activate the user's phone camera as the background of the app, we incorporated the Vuforia package [10] in Unity. This enables users to view their real-world environment through the camera, overlaid with virtual elements that appear naturally embedded within the scene. To render the app in stereoscopic view, necessary for head-mounted displays, we added the Google Cardboard [3] package. The app renders the scene using a single camera for efficiency, then splits the screen into two viewpoints—one for the left eye and one for

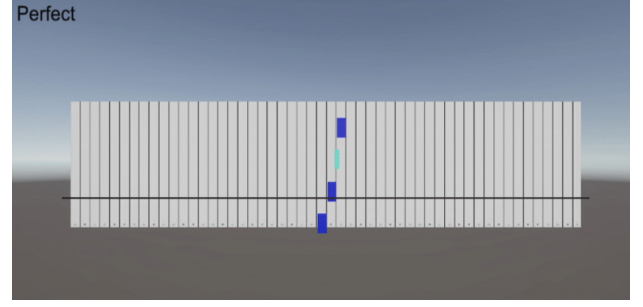


Fig. 6. Virtual screen mid-game



Fig. 7. Virtual screen at end of game



Fig. 8. Virtual screen spawned using image target

the right. As a result, when the user wears the headset, the two images merge into a cohesive 3D experience.

b) Image Target: We also utilized Vuforia's image target functionality to enable automatic alignment with the piano. This is done by creating a database of reference images that the app "scans" for; upon identifying a match, it places a virtual object at that position. To enable compatibility with different pianos, we thought to use a custom poster with the MUSIK logo that we can package and sell as part of the product. The customer affixes the poster on their piano, and the app recognizes the image, enabling it to produce the virtual screen precisely above the image. As opposed to specific piano models, using the poster makes our product universally compatible.

D. Accuracy Computation

To determine the type of feedback that must be provided to the user upon playing a note, an algorithm must be used that computes the accuracy. The algorithm can be split into the following two stages, which allows for any song with an exiting MIDI file to be used in the system.

- 1) The algorithm first calls a helper script named `midi2Tiles.py`, which is used to convert any given

MIDI file into a text readable format that can be used for the accuracy computation algorithm.

- 2) The algorithm titled `inputqueue.py` then begins a separate thread to take in input from the keyboard, starts a global timer, then compares the time at which a key is pressed (relative to a global time) to the time at which the key should be played. Using this information, it sends a timing message (“early”, “perfect”, “late”, “miss”).

1) *MIDI2Tiles Procedure*: This script is responsible for converting a given MIDI file to a text file with information containing when each note should be played in the song. In order to test the prototype, a demo MIDI file is used to test each of the keys in the one-octave C chromatic scale (to allow for all white and black keys between C4 and C5 to be tested).

The procedure first creates a `MidiFile` object the `mido` library, `mid = MidiFile(str)`. Next an empty list output is used to store the output structure, the MIDI file is converted to a dictionary object, and for each note in the dictionary, if the type of the note is either `note_on` or `note_off`, the note would be added to the dictionary (to extract only keyboard events). Finally a loop iterates through the dictionary structure, and if a `note_on` message appears, create a new list from this note to store into the output as a nested list of lists:

```
mem2=[]
if i['type'] == 'note_on':
    mem2.append(i['type'])
    mem2.append(i['note'])
    mem2.append(i['time'])
    mem2.append(i['channel'])
    output.append(mem2)
```

At the end, the following list structure is generated by this script, which will then be used by the Input Queue program as a reference for when notes should be played:

```
[['note_on', 60, 0, 0],
 ['note_on', 61, 0.6, 0],
 ['note_on', 62, 1.2, 0],
 ...]
```

Observe that in this structure, each entry contains the following information, which will reflect the data that is also collected from the keyboard while operating.

```
data = [event type, note number, velocity, 0]
```

Also observe that for the sake of simplicity, only key presses are detected in this algorithm, but in the future, the length at which notes are held may also be detected in a similar way through employing the `note_off` messages (see Results and Discussion).

2) *Input Queue*: This is the script we run while the user is playing to collect MIDI inputs from the piano and compare them with the example MIDI file. We also compute user accuracy and write them to the Arduino, which will be sent to

Event type	Masked value
Note pressed	0x90
Note released	0x80

Fig. 9. MIDI Event Types

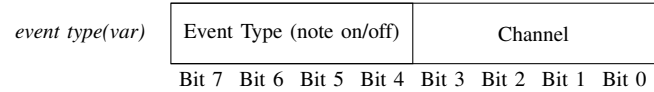


Fig. 10. Bit structure of event type

Unity at the end of the game. Currently our computer is acting as a middle-man between Unity and the Arduino, where we would want to eventually do all the processing on the Arduino.

a) *MIDI from Piano to Computer*: The `pygame` library is employed to capture MIDI data from our piano keyboard. We had to first install a driver from the Yamaha website so that our computer could sufficiently recognize the MIDI input and translate it to our operating system. Once the keyboard is connected to the computer via a USB port, the MIDI module within `pygame` is used to identify MIDI devices and monitor the port for incoming MIDI data. This process is encapsulated within a function that executes in a continuous loop.

Our method that detects MIDI inputs runs in a background thread, continuously polling the MIDI device for any new events. When a MIDI event is detected, the data is stored in a two-element Python list with the following structure (note that data is of the same format as of the array structure generated by the `Midi2Tiles` procedure):

```
event = [data, timestamp]
```

```
data = [event type, note number, velocity, 0]
```

We determine whether a note has been played by the element *event type* in the data array (`data[0]`). *Note number* corresponds to a key number on the piano, and *velocity* is an integer from 0-255 representing how quickly the note was pressed. Fig. 9 displays the two types of events and their corresponding hexadecimal values that we use to detect the start and end signal of a note being played. Fig. 9 also mentions that these are the masked values; this is relevant because of the structure of the *event type* data.

From Fig. 10, the *event type* variable has 8 bits, where the upper 4 bits represent the event and the lower 4 bits represent the channel. Therefore, 0x90 and 0x91 both represent “Note On” events but in different channels. To combat this, we employ *masking*, where we effectively remove channel information and isolate event type. We also need to confirm that `velocity > 0` because of the different ways in which the on/off signals are sent. An *event type* = 0x90 + `velocity` = 0 actually indicates a “Note Off” event, whereas *event type* = 0x90 + `velocity` > 0 indicates a “Note On” event. Our final “if” statement to check if a note has been pressed is then:

```
if event type & 0xF0 == 0x90 and velocity > 0
```


where "& 0xF0" is how we mask the *event type* variable. Following the detection of note pressing, we convert *note number* (the MIDI note number) to a piano note number to be able to compare it to our sample MIDI file. All this requires is subtracting 20 from the MIDI note number. We then add this piano note number to a Queue structure to be used in accuracy computations.

b) *User MIDI and Sample MIDI Comparison*: In the main thread of our program, we will be calculating how accurate the users playing is. We have currently set a threshold of 200ms to account for error (since we can not expect them to play it *exactly* at the right time), which should be modified depending on the song speed. We index through the converted sample MIDI file (which is an array of note data) and compare it to the users pressed notes. Our current logic works as such:

- 1) Continuously loop through array till you reach the end
- 2) If the user misses the note (no note has been playing after 1.5 seconds), increment the indexing variable. 1.5 seconds is a temporary margin used for complete note misses, which would also have to be modified depending on the song.
- 3) If a note has been played, check the following
 - If the correct note was pressed before the *expected time - threshold*, denote the action as early. Note that expected time is given from the sample MIDI array
 - Else, if the correct note was pressed after the *expected time + threshold*, denote the action as late
 - Else, if the correct note was pressed, denote it as perfect
 - Otherwise if the wrong note was pressed, denote it as a miss
- 4) If a note has been played 1.5 seconds early, do not increment the indexing variable

Once the end of the song has been reached, send the final score results to the Arduino so they can be displayed on Unity.

III. RESULTS AND DISCUSSION

Our prototype greatly reduced the originally cumbersome MUSIK 1.0 design and made playing more accessible to users by only requiring a few key components. The game screen that we created effectively displays piano notes and live player feedback. The user is able to see the tiles falling in time to the music and play the song and receive accurate timing feedback. The AR/VR headset proved to be effective in immersing the user in the AR world. The headset was inexpensive to make, making our product accessible for all users.

One critical part of the implementation that required a large amount of experimentation and tuning was in the timing thresholds. In particular, while the algorithm sequentially iterates through each note to evaluate timing, eventually if an individual is very late in pressing the note, so much so that the note is no longer on the screen, the note should immediately be considered late and the algorithm advance to the next note in the song. Similarly, if an individual is very early in pressing the note, so that the note is nowhere near the judgment line, the note should be considered early but the algorithm should

Timing Result	Relative Allowable Times
Early, do not advance	$t < -1.5$
Early	$-1.5 \leq t < -0.2$
Perfect	$-0.2 \leq t \leq 0.2$
Late	$0.2 < t \leq 1.5$
Late, skip	$t > 1.5$

Fig. 11. Timing Results for Key Press Times

not advance to the next note as the user may want to reattempt the note.

These thresholds had to be closely evaluated, and eventually one set of thresholds that worked with the design is shown in Fig. 11. However, these may not necessarily be the most optimal, such as for faster songs where like 'ChromaticScale.mid' where notes arrive at intervals of 0.6 seconds. One method to remedy this issue may be to find where the fastest note difference occurs and adjust thresholds accordingly, but may become difficult for individuals to adjust to different thresholds depending on the song. As such, this tradeoff must still be considered when tuning these timing thresholds according to the song between leniency and accuracy.

A. Future Work and Improvements

With more time, the following may be integrated:

a) *Features*: Add the ability to upload a song of choice, customize song speeds, and pause/play/restart buttons. This would help users learn more difficult parts of the songs and greatly improve practice capabilities.

b) *Headset*: The physical prototype was sturdy and light, but it would benefit from a few changes. The lenses were slightly too far from the phone, resulting in blurry vision. With more time, we would be able to further test and find the precise distance required to provide the best visual experience. Furthermore, we could implement adjustability for the phone housing and lens distance to accommodate different sizes of phones and face shapes.

In terms of manufacturing, we could experiment with finding efficient ways to assemble the headset for mass production, as adhesives take a long time to cure. Developing and implementing snap-fit mechanisms for assembly could accomplish this. Lastly, we would consider different materials and take advantage of their unique properties. For example, we could use a more durable material such as polyethylene terephthalate glycol (PETG) instead of PLA to prevent the headset components from snapping.

c) *Note Off Timings*: Our accuracy computations only factor when the user presses a key. With more time we could revise our accuracy code to incorporate when they release the key to widely increase the scope of their playing. This implementation would add another layer of learning to play piano songs and greatly improve use.

d) *Arduino Computations*: The computer currently does all of the accuracy computations and acts as a middle-man during some signal transmission between Unity and the Arduino. In the future we would prefer to eliminate the need of a computer and instead doing all the computations on the

Arduino. This would also make our product more accessible, no longer requiring users to have a computer and phone.

e) Visual Effects: Our visualizer displays the notes and accuracy feedback, but adding visual effects (similar to Guitar Hero) would greatly increase user enjoyment and how often they continue practicing.

f) AI Instructor: Currently the user receives live timing feedback, but the user might benefit from an AI instructor that can give more feedback on their playing

IV. CONCLUSIONS

Overall, MUSIK 2.0 presents a significant contribution in the context of design and development through its effectiveness in being both an immersive and simple to set up experience, which both contribute to its accessibility for all individuals to learn and especially in improving learning piano at home. Our design demonstrates how a full AR system may be successfully implemented with only a smartphone, and is highly flexible in allowing for any song or piano to be utilized in the operation of the system. By lowering the barrier to being a self-contained system and teaching through a visual experience, it motivates individuals to learn more through practicing enjoy and find a more enjoyable experience playing the piano.

ACKNOWLEDGEMENTS

We would like to thank Dean Antoine, Megan Holleran, our design advisor Sahana Ranganathan, our alumni mentor Rahul Hegde, and the rest of the design advisor team for guiding this project to where it is today.

REFERENCES

- [1] "AR Mobile Template Quick Start Guide — AR Mobile — 1.0.1." Unity3d.com, 2024, docs.unity3d.com/Packages/com.unity.template.ar-mobile@1.0/manual/index.html.
- [2] Dobroselsky, Maxim. "Melanchall/Drywetmidi." GitHub, 24 July 2023, github.com/melanchall/drywetmidi.
- [3] "Google Cardboard for Unity." Google Developers, developers.google.com/cardboard/develop/unity/quickstart
- [4] "How to Add a Visualizer to Your Piano (after Effects Tutorial)." YouTube, HalcyonMusic, 26 Aug. 2023, www.youtube.com/watch?v=5ZXz5cORqrA.
- [5] "Piano Key Frequencies." Wikipedia, Wikimedia Foundation, 20 Mar. 2024, en.wikipedia.org/wiki/Piano_key_frequencies.
- [6] "Pygame Documentation." Pygame Front Page - Pygame v2.6.0 Documentation, www.pygame.org/docs/.
- [7] SkyanSam. "GitHub - SkyanSam/RhythmGameTutorialProject" GitHub, 2021, github.com/SkyanSam/RhythmGameTutorialProject.
- [8] "Specifications for viewer design - Cardboard Manufacturer Help," Google.com, 2019, https://support.google.com/cardboard/manufacturers/answer/6323398
- [9] Vandenneucker, Dominique. "MIDI Tutorial." Arpege Music - Dominique Vandenneucker, www.cs.cmu.edu/music/cmsip/readings/MIDI%20tutorial%20for%20programmers.html.
- [10] "SDK Download — Vuforia Developer Portal." Developer.vuforia.com, developer.vuforia.com/downloads/sdk.